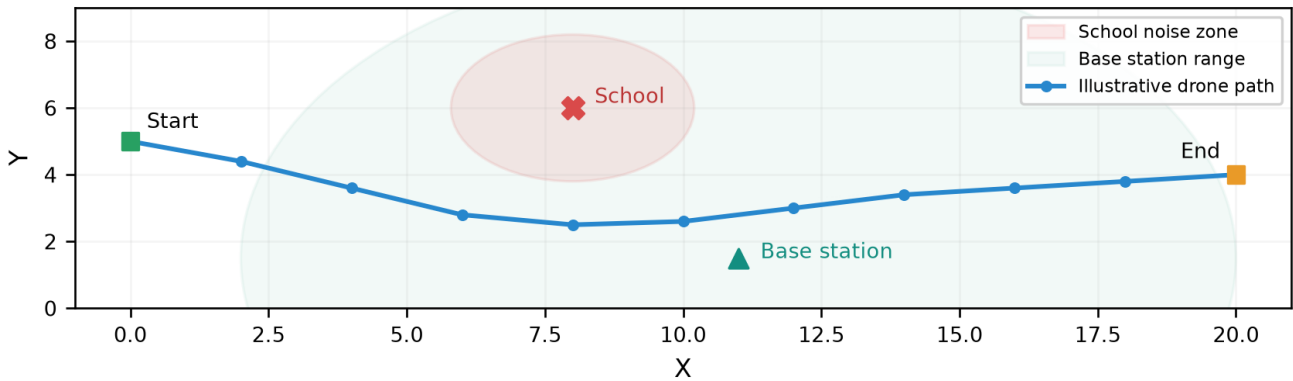


Task: Drone Path Optimization (Discrete Points)



Illustrative path with one school. The optimizer selects intermediate Y-coordinates.

Overview

You are given a fixed **start point** and **end point** in a 2D plane. Your task is to plan an optimal flight path for a drone by choosing a sequence of lateral positions at evenly spaced milestones along the horizontal axis. The path is represented as a polyline connecting these waypoints.

The challenge is to balance three competing objectives: **flight efficiency**, **noise mitigation**, and **signal stability**.

Input

Parameter	Description
Start point (x_0, y_0)	The fixed takeoff location.
End point (x_e, y_e)	The fixed landing location.
num_points	The number of intermediate waypoints you must specify. The X -coordinates of these waypoints are fixed and evenly spaced between x_0 and x_e ; you only control their Y -coordinates.
School coordinate (x^s, y^s)	The single school location for this case, passed as <code>school=(x_s, y_s)</code> . The drone should avoid flying close to it.
Base station coordinate (x_b, y_b)	Location of the signal base station. The drone must stay within reliable range.

Note: The X -coordinates of all intermediate waypoints are determined automatically by the system. You are only required to output the corresponding Y -coordinates.

Output

A list of Y -coordinates of length `num_points`:

$$[y_1, y_2, \dots, y_n], \quad n = \text{num_points}$$

The evaluator converts each returned value to a float and clamps it to $[-50, 50]$ before scoring. The list must contain exactly `num_points` values; an incorrect length or a conversion/evaluation error makes the case invalid.

Together with the auto-assigned X -coordinates, these define the intermediate waypoints $p_i = (x_i, y_i)$ for $i = 1, \dots, n$. The full path is the polyline:

$$\text{start} = p_0 \rightarrow p_1 \rightarrow \dots \rightarrow p_n \rightarrow p_{n+1} = \text{end}$$

Scoring

For each test case, the evaluator computes the three normalized factors defined below. It then averages each factor across the eight test cases, which use 10-100 intermediate waypoints, and calculates the recorded **Score** as:

$$\text{Score} = -\overline{F_1} \times \overline{F_2} \times \overline{F_3}$$

A higher Score is better. The standalone evaluator calls the positive product `final_score`, while ShinkaEvolve records its negative as `combined_score`. Maximizing the recorded Score is therefore equivalent to minimizing the product of the mean factors. A poor performance on any single factor will drag down the overall Score.

Factor Definitions

Let $d(p, q)$ denote the Euclidean distance between points p and q .

F1 - Flight Efficiency

Measures how much longer your path is compared to a straight line. The optimal straight-line path has $F_1 = 1$; any detour increases it.

$$f_1 = \sum_{i=0}^n d(p_i, p_{i+1}), \quad F_1 = \frac{f_1}{d(\text{start}, \text{end})}$$

F2 - Noise Penalty

Penalizes the drone for flying close to the case's school. The penalty for each waypoint grows sharply as the drone approaches it.

$$f_2 = \sum_{i=1}^n \frac{1000}{(x_i - x^s)^2 + (y_i - y^s)^2 + 10}, \quad F_2 = \frac{f_2}{n}$$

Each test case contains exactly one school at (x^s, y^s) .

F3 - Signal Risk

Penalizes the drone for straying too far from the base station, which increases the risk of losing signal.

$$f_3 = \sum_{i=1}^n d((x_i, y_i), (x_b, y_b)), \quad F_3 = \frac{f_3}{n}$$

Key Observations

- **Trade-offs are unavoidable.** The path that minimizes distance (F_1) is a straight line, but the straight line may pass over the school (raising F_2) or drift far from the base station (raising F_3).
- **The multiplicative structure is unforgiving.** A path with one catastrophically bad factor (e.g., flying directly over a school) will result in a very poor Score, regardless of how well it performs on the other two factors.
- **A good solution balances all three factors simultaneously.**